



Data Pipeline & Data Observability

AICB-P1 · Garbage in → garbage out — fix thế nào?

Tên Giảng Viên

VinUniversity · Phase 1 · 2026

HÃY SUY NGHĨ...

“Agent của bạn dùng data từ database công ty. Đột nhiên data sai – agent hallucinate. Bạn có biết không?”

Giữ câu hỏi này trong đầu khi học bài hôm nay

1. Data Pipeline Fundamentals
2. Ingestion – Thu Thập Data Từ Nhiều Nguồn
3. Transform – Làm Sạch & Chuẩn Hóa Data
4. Data Quality – 6 Dimensions
5. Data Observability
6. ETL Automation & Orchestration

- **60–80% thời gian** trong AI project thực tế là data work – không phải model
- Một agent RAG xuất sắc vẫn **hallucinate** nếu vector store được nạp data bẩn
- **Garbage in → garbage out**: quality của output tỷ lệ thuận với quality của input data
- Observability = cơ chế phát hiện data sai **trước khi user phàn nàn**

Thực tế dự án AI:

20% – xây model/agent

80% – data collection, cleaning, pipeline, monitoring

Agenda: Pipeline Fundamentals → Ingestion → Transform for AI → Quality Gates → Observability & Debugging → Orchestration → Lab

Data Pipeline Fundamentals

Hiểu chuỗi xử lý từ nguồn đến agent – ETL, ELT, Batch, Streaming

Data Pipeline — Chuỗi các bước tự động hóa việc **thu thập**, **xử lý**, và **phân phối** data từ nguồn đến đích

AI Data Stack điển hình:



Sources: DB, API, files, streams

Pipeline: ingest + transform

Storage: warehouse, vector store

Serving: API, cache layer

Agent: LLM + tools + RAG

Layer	Tools 2026
Ingestion	Airbyte / Fivetran (managed) + Debezium (CDC, chuẩn mở)
Storage	Lakehouse: S3/GCS + Iceberg hoặc Delta Lake (chi tiết: Day 18)
Transform	dbt (+ Fusion engine mới, viết lại bằng Rust) – contracts enforce schema
Orchestration	Airflow 3.0 (event-driven scheduling) hoặc Dagster (asset-centric, lineage sẵn có)
Observability	Monte Carlo / Elementary + OpenLineage (chuẩn interoper bên dưới)
Activation (<i>optional</i>)	Reverse ETL (Census/Hightouch) – giống Serving→Agent, nhưng cho CRM

Fivetran mua Census + sáp nhập **dbt Labs**; Datadog mua Metaplane – thị trường đang gộp lại quanh vài platform lớn.

Scenario: Agent trả lời câu hỏi về chính sách công ty, ticket hỗ trợ và SOP nội bộ.



- Nếu **ingestion fail**: tài liệu mới không vào store → agent trả lời cũ
- Nếu **transform sai**: chunk xấu, metadata thiếu → retrieve nhầm
- Nếu **index lỗi**: embed thiếu hoặc duplicate → context méo

Điểm khác với dashboard BI:

BI sai → số sai trên báo cáo

Agent sai → hành động hoặc trả lời sai trực tiếp với user

Vì vậy pipeline cho AI cần thêm:
chunking, metadata, embeddings, retrieval checks, trace logs

Trước khi học tiếp: **ETL** và **ELT** khác nhau ở đâu? Mỗi loại thường dùng công nghệ/tool gì? Khi nào bạn sẽ chọn ETL, khi nào chọn ELT cho một hệ thống AI/agent?

Cá nhân – 8 phút tự tìm hiểu/nhớ lại (dùng phone hoặc kiến thức sẵn có) + 1–2 bạn chia sẻ.

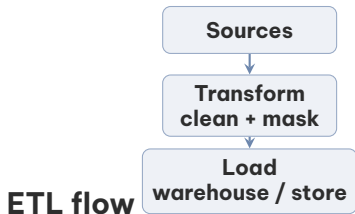
Gợi ý nếu bí: nghĩ theo hướng “transform trước hay sau khi lưu”.

ETL (Extract → Transform → Load)

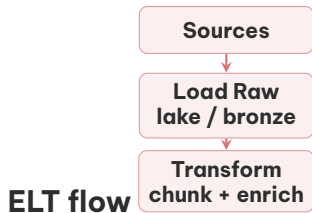
- Transform **trước** khi load vào kho
- Phù hợp: data nhạy cảm, cần mask trước khi lưu
- Ví dụ: redact PII trong ticket support trước khi embed cho agent
- Tools: Talend, Informatica, custom scripts

ELT (Extract → Load → Transform)

- Load raw data, transform **sau** trong kho
- Phù hợp: big data, cloud data warehouses
- Ví dụ: load raw docs/logs trước, rồi chunk + enrich trong lakehouse
- Tools: Spark SQL, BigQuery, custom Python jobs



- Dùng khi cần **lọc/ràng buộc trước** khi lưu
- Hợp với data nhạy cảm, dữ liệu production cho agent
- Ví dụ: redact PII rồi mới tạo embeddings



- Dùng khi cần **giữ raw** để replay, backfill, thử nghiệm
- Hợp với RAG/ML có nhiều nguồn và logic transform thay đổi liên tục
- Ví dụ: lưu raw docs trước, sau đó thử nhiều chiến lược chunking

Chọn ETL nếu

- cần mask PII trước
- schema khá ổn định
- data đi vào agent phải rất sạch
- muốn giảm rủi ro lưu raw nhạy cảm

Chọn ELT nếu

- nhiều nguồn, nhiều định dạng
- phải backfill / replay thường xuyên
- còn đang thử chunking, labeling, feature engineering
- cần giữ raw cho audit và experiment

Thực tế

Nhiều team dùng **hybrid**:

Load raw trước, nhưng **ETL các phần nhạy cảm** như PII, secrets, dữ liệu pháp lý trước khi index hoặc serve cho agent.

Batch Processing

- Xử lý theo lô, theo lịch (hourly/daily)
- **Ưu:** đơn giản, cost thấp, dễ debug
- **Nhược:** latency cao (data trễ vài giờ)
- Dùng khi: training data, daily reports, ETL

Streaming Processing

- Xử lý realtime khi data xuất hiện
- **Ưu:** latency thấp (ms-giây)
- **Nhược:** phức tạp hơn, cost cao hơn
- Dùng khi: fraud detection, live agent context

Kafka và các lựa chọn thay thế:

- **Kafka:** vẫn là incumbent, ecosystem lớn nhất
- **Redpanda:** Kafka-API-compatible, viết bằng C++, ops đơn giản hơn, chi phí thấp hơn
- **WarpStream:** Kafka-API-compatible, lưu trên object storage, tính phí theo usage
- Confluent (chủ Kafka) đang chuyển hướng messaging sang **Apache Flink** như chuẩn stream processing

Data quality cho streaming:

- **Schema Registry:** chặn event sai format ngay lúc ingest, không đợi batch check
- **Flink / ksqldb:** viết rule kiểm tra realtime – giá trị bất thường, thiếu ID, volume spike
- Khác batch: không thể chạy Great Expectations suite hàng đêm trên stream vô hạn

Case study thật: Grab Engineering dùng FlinkSQL để convert data contracts thành rule kiểm tra realtime – chạy production, không phải lý thuyết.

Ingestion – Thu Thập Data Từ Nhiều Nguồn

Kết nối nguồn data đa dạng vào pipeline một cách đáng tin cậy

Structured sources

- **Databases** (PostgreSQL, MySQL): CDC để capture changes
- **Data warehouses**: Snowflake, BigQuery
- **REST / GraphQL APIs**: rate limits cần xử lý

Unstructured sources

- **Files**: CSV, JSON, Parquet, PDF, Word
- **Object storage**: S3, GCS, Azure Blob
- **Web scraping**: HTML → text extraction

Event streams

- **Kafka / Kinesis**: high-throughput event bus
- **Webhooks**: push từ external services
- **IoT sensors**: time-series data

CDC – Change Data Capture – detect & capture mọi INSERT/UPDATE/DELETE trong database để sync realtime thay vì full scan. **Debezium** (build trên Kafka Connect) là tool mã nguồn mở chuẩn phổ biến nhất.

Trong hệ AI/agentic, ingestion thường lấy từ:

- **Knowledge sources:** Notion, Confluence, PDF, Word, SharePoint
- **Transactional data:** CRM, ticketing, order DB, HR systems
- **Logs + feedback:** chat transcripts, tool calls, thumbs up/down, escalation notes

Thiết kế ingestion tốt cần:

- **Incremental sync:** chỉ lấy phần changed since last run
- **Idempotent upsert:** chạy lại không tạo duplicate chunks
- **Source versioning:** biết bản nào mới nhất, sync lúc nào

Rate limiting: source API giới hạn req/min → cần exponential backoff

Backpressure: consumer xử lý chậm hơn producer → cần buffer hoặc pause signal

Retry logic: dead-letter queue cho failed records

Thực tế AI: 1 file sync fail có thể khiến policy mới không tới agent

Câu hỏi user: “Chính sách hoàn tiền mới nhất là gì?”

Agent cần data từ nhiều nguồn:

- **CRM:** đơn hàng, trạng thái giao dịch
- **Policy docs:** chính sách hoàn tiền theo từng tháng
- **Ticket history:** case tương tự đã được xử lý ra sao
- **Escalation notes:** khi nào agent phải chuyển người thật

Nếu ingestion thiếu **1 nguồn quan trọng**, agent có thể:

- trả lời bằng policy cũ
- không biết ngoại lệ business
- đề xuất hành động không đúng với trạng thái đơn hàng

Checklist ingestion cho AI:

1. Có lấy **đúng nguồn** không?
2. Có lấy **đủ bản mới nhất** không?
3. Có biết record nào **thất bại** không?
4. Có log được **run ID** và thời gian sync không?

Transform – Làm Sạch & Chuẩn Hóa Data

Biến raw data thành data agent có thể tin tưởng và sử dụng được

- **Missing values:** NULL, empty string, “N/A” – drop, impute, hoặc flag
- **Outliers:** giá trị bất thường ảnh hưởng embedding quality
- **Duplicates:** cùng record xuất hiện nhiều lần → dedup bằng hash hoặc fuzzy match
- **Wrong formats:** date “31/12/2024” vs “2024-12-31” → standardize
- **Encoding issues:** UTF-8 vs Latin-1 → luôn enforce UTF-8

Text normalization cho AI:

- **Lowercasing:** tùy model, không phải lúc nào cũng cần
- **Unicode normalization:** NFC/NFD cho tiếng Việt
- **Whitespace:** collapse multiple spaces, strip trailing
- **HTML stripping:** loại bỏ tags trước khi embed
- **Language detection:** tách chunks theo ngôn ngữ

Schema validation: enforce data contracts – reject records không đúng schema thay vì để lọt vào model

dbt (data build tool) – SQL transformation được version control, test và document – biến SQL thành software engineering workflow

Tại sao dbt quan trọng:

- **Modularity:** mỗi transform là một .sql model riêng
- **Lineage:** tự động sinh DAG dependency graph
- **Testing:** built-in tests (not-null, unique, accepted-values)
- **Documentation:** auto-generate data catalog
- **Version control:** PR review cho data logic

```
-- models/cleaned_docs.sql
WITH raw AS (
  SELECT * FROM {{ ref('raw_documents') }}
),
cleaned AS (
  SELECT
    id,
    TRIM(LOWER(content)) AS content,
    created_at::date AS doc_date
  FROM raw
  WHERE content IS NOT NULL
    AND LENGTH(content) > 50
)
SELECT * FROM cleaned
```

Ý chính – BI thường transform để báo cáo; AI transform để model **hiểu đúng ngữ cảnh** và **retrieve đúng evidence**

Các bước transform thường gặp trong AI:

- **Clean text:** bỏ HTML, ký tự lỗi, OCR noise
- **Chunking:** chia tài liệu thành đoạn vừa ngữ nghĩa, vừa token budget
- **Metadata enrichment:** gắn source, owner, version, effective date
- **Redaction:** loại PII/secrets trước khi embed
- **Canonicalization:** chuẩn hóa tên sản phẩm, mã đơn hàng, timestamp

```
doc = load_pdf("refund-policy.pdf")
text = clean_text(doc.text)
chunks = chunk(text, size=500, overlap=80)

for i, chunk in enumerate(chunks):
    write_record({
        "chunk_id": f"{doc.id}:{i}",
        "content": chunk,
        "source_doc": doc.id,
        "version": doc.updated_at,
        "department": "support"
    })
```

Chunk quá to:

- chứa nhiều chủ đề → retrieval mơ hồ
- tốn token, giảm chỗ cho reasoning

Chunk quá nhỏ:

- mất context quan trọng
- câu trả lời thiếu điều kiện hoặc ngoại lệ

Metadata tốt giúp agent:

- filter theo phòng ban, ngày hiệu lực, loại tài liệu
- hiển thị citation đúng nguồn
- trace ngược về document gốc khi có lỗi

Chunk tốt thường cần

content
chunk_id
source_doc_id
section / title
effective_date
owner / department
version / updated_at

Lưu ý: Nhiều team chỉ embed “text thuần” mà quên metadata – retrieve đúng đoạn nhưng không biết nó từ bản policy nào.

Data Quality – 6 Dimensions

Đo lường chất lượng data trước khi nó đến tay agent

1. Completeness

Không thiếu records hoặc fields quan trọng.
Check: % NULL, row count so với expected

2. Accuracy

Data đúng với thực tế. Check: validate với nguồn gốc, business rules

3. Consistency

Cùng entity, cùng format across systems.
Check: cross-system reconciliation

4. Timeliness

Data đủ fresh cho use case. Check: max age, last-updated timestamp

5. Validity

Data theo đúng format và domain rules.
Check: regex patterns, range checks

6. Uniqueness

Không có duplicates. Check: dedup rate, composite key uniqueness

Great Expectations – Framework Python để viết, run và document data quality checks – “expectations” là assertions về data

Workflow cơ bản:

1. **Profile** data: tự động suggest expectations
2. **Write** expectations: not-null, unique, in-range, regex
3. **Validate** trước khi data vào pipeline
4. **Report**: HTML data docs tự động sinh
5. **Alert**: fail pipeline nếu expectations không pass

```
import great_expectations as gx

context = gx.get_context()
batch = context.sources.pandas_default\
    .read_csv("docs.csv")

batch.expect_column_values_to_not_be_null(
    "content"
)
batch.expect_column_value_lengths_to_be_between(
    "content", min_value=50
)
results = batch.validate()
print(results["success"]) # True / False
```

Ý chính – Trong AI pipeline, data quality không chỉ bảo vệ warehouse mà còn bảo vệ **retrieval**, **tool use** và **final answer**

Các quality gates nên có:

- **Schema gate:** có đủ content, doc_id, updated_at
- **Freshness gate:** policy quá cũ thì reject hoặc cảnh báo
- **Content gate:** text đủ dài, OCR confidence không quá thấp
- **Dedup gate:** cùng chunk không được nạp nhiều lần
- **PII gate:** không embed số thẻ, mật khẩu, access token

```
def validate(record):  
    assert record["content"].strip()  
    assert record["updated_at"] >= cutoff_date  
    assert len(record["content"]) >= 80  
    assert not contains_secret(record["content"])  
    assert record["chunk_id"] not in seen_ids  
  
for record in cleaned_records:  
    validate(record)
```

Data issue

- Missing documents
- Outdated version
- Duplicate chunks
- Wrong metadata
- Secret leakage

Agent symptom

- không tìm thấy bằng chứng liên quan
- trả lời dựa trên policy cũ
- lặp lại cùng một ý nhiều lần
- cite sai phòng ban / sai ngày hiệu lực
- làm lộ dữ liệu nhạy cảm cho user

Điểm dạy học quan trọng: nhiều lỗi nhìn giống “model hallucination” nhưng gốc rễ thực ra là data pipeline bug.

Data Observability

Monitor, alert và debug data problems trước khi agent bị ảnh hưởng

05

Agent RAG của bạn đang trả lời sai – thông tin cũ, khách hàng phàn nàn. Bạn có 10 phút, agent này **không có logging/observability nào cả**. Bạn sẽ kiểm tra những gì đầu tiên để tìm nguyên nhân? Liệt kê ý tưởng của bạn.

Cá nhân – 8 phút brainstorm + 2–3 bạn chia sẻ ý tưởng.

Gợi ý nếu bí: nghĩ theo hướng “data có mới không, có đủ không, có lỗi gì không”.

Freshness – Data có đang được update theo đúng lịch?

Distribution – Giá trị phân phối có bất thường không? (null rate, range)

Volume – Số lượng records tăng/giảm bất thường?

Schema – Cột bị đổi tên, thêm, xóa không?

Lineage – Data đến từ đâu, đi qua transform nào?

Data Lineage – Track hành trình data từ nguồn gốc → pipeline → chunk/index → retrieved context → model output

Muốn debug được, phải log ít nhất:

- question / session ID
- retrieved chunk IDs
- source document version
- embedding/index version
- pipeline run ID

Scenario: RAG agent trả lời sai

1. User phàn nàn agent đưa thông tin cũ
2. Check **answer trace**: agent đã retrieve chunk nào?
3. Check **Freshness**: chunk đó thuộc policy version ngày nào?
4. Check **Volume**: số documents embed hôm nay có drop về 0 không?
5. Trace **Lineage**: ingestion run 2am fail ở bước sync policy
6. Root cause: API timeout, retry/backoff chưa cấu hình đúng

Monitoring metrics cần theo dõi:

- **Pipeline SLA**: % runs hoàn thành đúng giờ
- **Row count delta**: Δ records qua các runs
- **Null rate per column**: alert nếu tăng đột biến
- **Schema drift**: tự động detect column changes
- **Data freshness**: max age của records trong store
- **Embedding coverage**: % chunks đã được embed

Không có observability: phát hiện sau 8 giờ, 500 users bị ảnh hưởng

Quy trình debug nên đi theo 5 lớp:

1. **Output layer:** agent trả lời gì, cite gì, confidence ra sao?
2. **Retrieval layer:** top-k chunks nào được lấy ra? có zero-hit không?
3. **Index layer:** chunk đó được embed bằng model/version nào?
4. **Pipeline layer:** run nào sinh ra chunk? pass/fail quality gates nào?
5. **Source layer:** tài liệu gốc có đúng, mới và đầy đủ không?

Lưu ý: Nếu bạn chỉ nhìn final answer mà không trace được về chunk và source document, bạn đang debug trong bóng tối.

Fields nên có trong trace log:

- request_id
- pipeline_run_id
- retrieved_chunk_ids
- source_doc_ids
- source_version
- embedding_model
- latency_ms
- fallback_used

Pipeline / data signals

- freshness của knowledge base
- failed sync count, dead-letter queue size
- duplicate chunk rate
- missing metadata rate
- embedding queue lag

Agent / product signals

- retrieval hit rate
- % answers có citation hợp lệ
- user correction / escalation rate
- tool-call failure rate
- abandoned conversations sau câu trả lời sai

Quan điểm thực chiến: observability tốt phải nối được **data issue** → **retrieval issue** → **business impact**.

Trace & Span – Trace = một lần agent chạy end-to-end; Span = một bước bên trong (1 LLM call, 1 tool call, 1 retrieval) – cùng ý tưởng freshness/volume/schema bạn vừa học, chỉ khác tầng đo: model thay vì pipeline

Kiến trúc thực tế:

- Agent call → instrument bằng OpenTelemetry GenAI spans
- Gửi trace về **Langfuse** (self-host, open-source), **Phoenix** (open-source, OpenInference), hoặc **LangSmith** (managed, free tier 5k trace/tháng)
- Score tự động bằng **RAGAS** (faithfulness, context precision/recall)
- Alert khi quality hoặc cost drift

Lưu ý chọn tool: Helicone đã bị Mintlify mua (03/2026), giờ chỉ maintenance mode – không còn phát triển tính năng mới. Ưu tiên Langfuse/Phoenix nếu cần self-host lâu dài.

Mỗi trace ghi lại:

- Prompt gửi đi, response nhận về
- Token usage, latency, cost
- Từng tool/retrieval step lồng bên trong (nested spans)

Agent trả lời sai → mở trace → xem span nào chậm/sai: retrieval hay generation?

```
from langfuse import Langfuse
langfuse = Langfuse()

trace = langfuse.trace(
    name="rag-query", input={"question": q}
)
retrieval = trace.span(name="retrieval")
chunks = vector_store.search(q, top_k=5)
retrieval.end(output={"chunk_ids":
    [c.id for c in chunks]})

generation = trace.generation(
    name="generation", model="gpt-4o-mini",
    input=chunks,
    usage={"input": 512, "output": 128},
)
```

Vector store health:

- Qdrant / Weaviate: expose Prometheus metrics (/metrics) – recall, latency, memory
- **Embedding drift:** retrieval quality giảm dù “embedding coverage” vẫn 100%

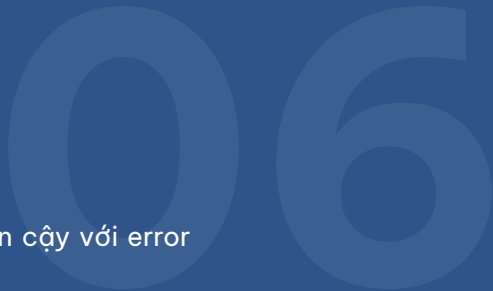
```
from ragas.metrics import (
    faithfulness, answer_relevancy
)
from ragas import evaluate

result = evaluate(
    dataset,
    metrics=[faithfulness, answer_relevancy]
)
# {'faithfulness': 0.83,
#  'answer_relevancy': 0.91}
```

Lưu ý: Cost/token là trục alert riêng: 1 agent loop lỗi có thể x100 chi phí trong vài phút mà không hề có anomaly ở row count hay schema – track \$/request và token delta giống cách bạn track row count delta.

ETL Automation & Orchestration

Đưa pipeline vào vận hành tự động, đáng tin cậy với error handling đúng chuẩn



Core concepts:

- **DAG** (Directed Acyclic Graph): định nghĩa thứ tự task
- **Operator**: đơn vị thực thi (PythonOperator, BashOperator, ...)
- **Scheduler**: trigger DAGs theo cron hoặc event – **Airflow 3.0** (2025) thêm event-driven native + DAG versioning
- **Executor**: chạy tasks (Local, Celery, Kubernetes)
- **XCom**: truyền data nhỏ giữa tasks

Khi nào dùng Airflow:

- Batch pipeline phức tạp với nhiều dependencies
- Team đã có Python skills
- Cần visibility đầy đủ qua UI

Modern alternatives:

Prefect

Python-native, ít boilerplate hơn Airflow. Flows = Python functions. Phù hợp team muốn nhanh.

Dagster

Asset-centric orchestration – model data assets, không phải tasks. Built-in lineage & observability. Phù hợp data-heavy teams.

Tool	Best for	2026 update
Airflow 3.0	Batch DAG phức tạp, ecosystem lớn	Event-driven scheduling + DAG versioning (mới – trước đây chỉ cron/deps)
Dagster	Asset-centric, cần lineage/observability sẵn có	Vẫn là lựa chọn nếu muốn lineage “miễn phí”
Prefect	Python-native, setup nhanh	Positioning không đổi
Mage AI	Low-code, onboarding nhanh	Entrant mới, đáng nhắc tên
Kestra	Workflow orchestration rộng hơn (không chỉ data asset)	Entrant mới, đáng nhắc tên

Airflow

Hay dùng cho:

batch ETL, retraining theo lịch, multi-step jobs

Lý do:

mature, nhiều operator, UI quen thuộc

Prefect

Hay dùng cho:

Python pipelines, startup teams, flows cần code nhanh

Lý do:

ít boilerplate, local-to-cloud dễ

Dagster

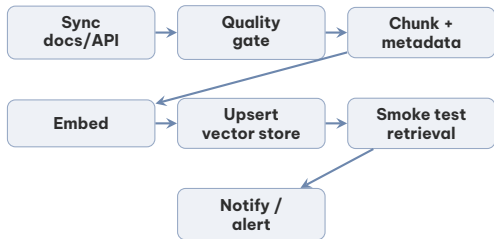
Hay dùng cho:

asset-heavy pipelines, lineage rõ, data platform teams

Lý do:

asset model hợp với tables, features, indexes

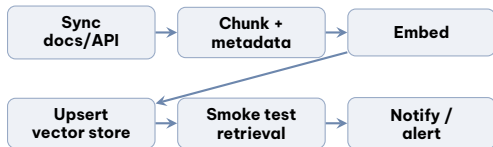
Góc nhìn thực tế: hệ RAG/agent nhỏ thường bắt đầu bằng cron + Python; khi số bước, số nguồn, và số team tăng lên thì mới nâng lên Airflow / Prefect / Dagster.



Cách dùng trong thực tế:

- **Trigger:** mỗi giờ, khi có file mới, hoặc khi policy đổi
- **Fail fast:** quality gate fail thì **không** cho index tiếp
- **Smoke test:** chạy vài câu hỏi chuẩn để check retrieval
- **Notify:** báo Slack nếu index mới làm hit rate giảm

Lưu ý: Trong AI pipeline, orchestration không chỉ “chạy jobs” mà còn kiểm soát **chất lượng đầu vào** trước khi agent dùng data mới.



Đây là pipeline production của một startup X. Nhìn kỹ – so với những gì ta vừa học, pipeline này đang thiếu bước quan trọng nào?

Điều gì sẽ xảy ra trong thực tế nếu một tài liệu bị lỗi (OCR hỏng, thiếu field) đi qua pipeline này?

Cá nhân – 8 phút quan sát/tìm lỗi + 2-3 bạn chia sẻ.

Scheduling strategies:

- **Cron-based:** 0 2 * * * = 2am mỗi ngày – đơn giản, predictable
- **Event-driven:** trigger khi file mới upload hoặc webhook nhận được
- **Dependency-based:** chỉ chạy khi upstream pipeline xong
- **Backfill:** chạy lại pipeline cho historical dates

Error handling patterns:

- **Retry với backoff:** attempt 1 → 30s → attempt 2 → 2m → ...
- **Dead Letter Queue:** failed records không bị mất, xử lý sau
- **Partial failure:** idempotent tasks để re-run an toàn
- **Alerting:** Slack/email khi pipeline fail
- **SLA breach:** alert khi pipeline trễ so với deadline

Lưu ý: Idempotency là bắt buộc: chạy lại pipeline 2 lần phải cho kết quả giống chạy 1 lần. Thiếu idempotency dẫn đến duplicate data trong vector store.

Mục tiêu: Build AI data pipeline hoàn chỉnh: thu thập raw docs, làm sạch, chunk, enrich metadata, embed và nạp vào vector store cho agent. Simulate data corruption để đo impact lên retrieval và câu trả lời.

Deliverable: (1) Pipeline script: raw → cleaned → chunked → embedded; (2) Quality gates cho schema/freshness/duplicates; (3) Trace log để debug agent answers; (4) So sánh response quality trước/sau fix data

Thời gian: 4 giờ (Vibe Coding 1.5h + Lab 2.5h)

Những ý chính cần nhớ trước khi sang bài tiếp theo

- 1 Data pipeline là **hệ tuần hoàn** của mọi AI product – agent mạnh đến đâu cũng vô dụng nếu data vào bị bẩn
- 2 Pipeline cho AI khác BI ở chỗ phải tối ưu cho **retrieval, context quality, citations** và khả năng debug agent
- 3 Data **quality gates** phải chặn thiếu dữ liệu, dữ liệu cũ, duplicate chunks, metadata sai và secret leakage
- 4 **Observability** tốt cho phép trace từ câu trả lời sai ngược về chunk, pipeline run và source document – và tiếp tục vào tận trace/span của model call

Guardrails & AI Safety

“Agent hoạt động đúng không có nghĩa là an toàn – cần lớp bảo vệ ở mọi cấp”

- Đọc: OWASP Top 10 for LLMs (owasp.org)
- Thực hành: Thêm input/output validation vào ETL pipeline từ Lab 10
- Suy nghĩ: Agent của bạn có thể bị poisoned data attack không?

1. **Hidden Technical Debt in Machine Learning Systems** – Sculley et al., Google, NeurIPS 2015. Giải thích tại sao 80% thời gian AI = data work. Kinh điển, đọc trước lớp (30 phút).
2. **Designing Data-Intensive Applications** – Martin Kleppmann. Nền tảng cực tốt để hiểu ingestion, streaming, idempotency, backpressure, và consistency trong hệ thống data.
3. **Designing Machine Learning Systems** – Chip Huyen. Góc nhìn production cho data pipelines, data quality, monitoring và feedback loops trong AI systems.

Hỏi & Đáp

Garbage in → garbage out. Bạn kiểm soát data quality bằng cách nào trong project của mình?



Cảm ơn!

Ngày tiếp theo: Guardrails & AI Safety

labs + source code: github.com/vbi-academy/aicb-phase1